arXiv:2606.02745v2 [cs.RO] 13 Oct 2026

SeeTraceAct: Visibility-Aware Latent Planning from Cross-Embodiment Demonstration Videos

Jaehyeon Son[1,2] **Junhyun Kim**[1] **Kyle Kam**[1] **Jeremiah Coholich**[1] **Seok Joon Kim**[1]
Jinhoo Kim[1] **Chris Dongjoo Kim**[2] **Jaemin Cho**[2,3] **Dieter Fox**[2,4] **Zsolt Kira**[1]

[1]Georgia Institute of Technology [2]Allen Institute for AI
[3]Johns Hopkins University [4]University of Washington

https://github.com/jaehyeon-son/SeeTraceAct

Abstract: Vision-language-action models (VLAs) are promising general-purpose robot policies, but adapting them to new tasks typically requires costly task-specific teleoperation data. As an alternative, we study one-shot demo-conditioned VLAs, where a robot policy is conditioned on a single demonstration video of an unseen task. We find that existing end-to-end approaches often struggle when successful execution requires precisely localizing small target regions. To address this limitation, we propose SEETRACEACT, a demo-conditioned VLA framework that encourages precise spatial grounding through visibility-aware prediction of future end-effector traces. To enable reproducible evaluation with cross-embodiment demonstrations, we introduce and release ROBOCASA-DC, a demo-conditioned extension of RoboCasa that pairs its robot trajectories with newly collected humanoid demonstrations. Experiments on ROBOCASA-DC and a real-world benchmark, where a Franka Panda arm is conditioned on human demonstrations, show that SEETRACEACT outperforms baselines, achieving the best success rate across all four ROBOCASA-DC settings and improving real-world average success by 9.5 percentage points.

1 Introduction

Vision-language-action models (VLAs) have recently emerged as a promising path toward general-purpose robotic agents by adapting pretrained vision-language models (VLMs) for robotic control [1, 2, 3]. However, deploying VLAs on new tasks typically requires post-training on expert robot trajectories collected through teleoperation. Because collecting teleoperated robot data requires specialized hardware and substantial human effort, it remains a central bottleneck for scaling VLAs across diverse tasks, embodiments, and environments [4, 5, 6].

A scalable alternative is to let an end user specify a new robot task by performing it once in front of a camera. This motivates one-shot demo-conditioned VLAs, where a policy is conditioned on a single human demonstration video of an unseen target task. However, this problem is non-trivial: the cross-embodiment gap makes directly translating human demonstrations into robot actions fundamentally challenging.

While methods that utilize explicit embodiment-specific retargeting have been developed [7, 8, 9], they require embodiment- and task-specific expertise beyond what an end user can provide, so we instead focus on end-to-end approaches. Prior work in this direction has learned video representations and employed auxiliary objectives to ground demonstrations [10, 11, 12]. While these methods have shown promise, we find that they often struggle with precision-sensitive tasks, such as pressing the target button on a coffee machine or turning on a sink faucet (see §5.3).

To address this limitation, we propose SEETRACEACT, a demo-conditioned VLA framework that encourages precise spatial grounding through an auxiliary future-trace prediction objective (Fig. 1). Given a demonstration video, SEETRACEACT learns a *visual latent plan* that summarizes the task-

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

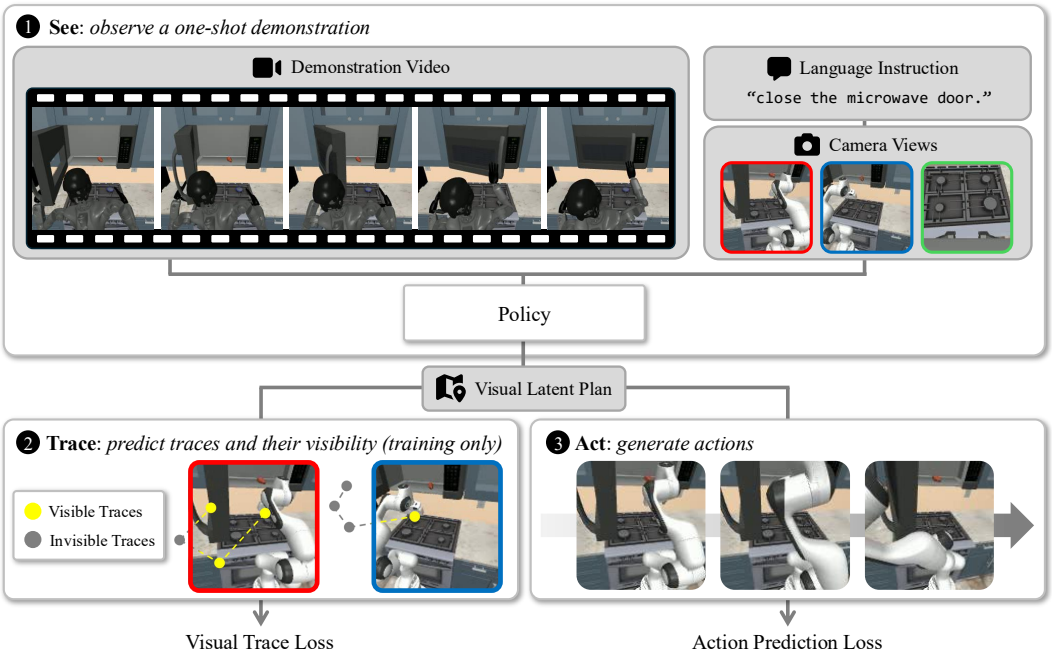


Figure 1: Overview of SEETRACEACT. Given a demonstration video, current camera views, and a language instruction, the policy encodes task-relevant information into a *visual latent plan* (SEE). During training, the policy predicts future visual traces and their visibility for each camera view (TRACE), while also predicting actions from the latent plan (ACT). At inference time, the trace prediction component is discarded, and the policy generates actions from the latent plan (ACT).

relevant motion the robot should execute. During training, this latent plan is decoded into future end-effector traces in the robot's camera views. However, trace supervision is not always well-defined, especially in multi-view setups: the end effector may move off-screen in some camera views, making its image coordinates ill-posed targets. SEETRACEACT therefore uses a visibility-aware trace decoder that predicts both trace coordinates and their validity, preserving supervision even when some trace points are off-screen. At inference time, the trace decoder is discarded, and the policy generates actions directly from the latent plan.

To support reproducible evaluation and future work on demo-conditioned VLAs, we introduce and release the ROBOCASA-DC benchmark, a demo-conditioned extension of the RoboCasa [13] simulation environment (§4). ROBOCASA-DC supports both same-embodiment and cross-embodiment evaluation, allowing policies to be conditioned on demonstration videos from either the Panda arm itself or a humanoid that serves as a proxy for human demonstrators. This complements prior evaluations that focus on real-world settings [10], use single-arm robot demonstrations in simulation [12], or do not release the human demonstrations used for simulated evaluation [11].

Experiments on ROBOCASA-DC and a real-world benchmark with a Franka Panda arm show that SEETRACEACT achieves the strongest results among competitive demo-conditioned VLA baselines (§5.2). On ROBOCASA-DC, SEETRACEACT achieves the best success rate across all four evaluation settings. On the real-world benchmark, where the robot is conditioned on human demonstrations, SEETRACEACT improves the average success rate by 9.5 percentage points. Our ablation study (§5.4) supports the importance of our key design choices, including visibility-aware trace supervision. Our main contributions are as follows:

1. We introduce SEETRACEACT, a demo-conditioned VLA framework that encourages precise spatial grounding of one-shot demonstration videos, including cross-embodiment ones, via visibility-aware prediction of future end-effector traces.

2. We introduce and release RoboCasa-DC, a demo-conditioned extension of RoboCasa for benchmarking policies with both same- and cross-embodiment demonstration videos.
3. We show that SeeTraceAct achieves the strongest results among competitive baselines on RoboCasa-DC and a real-world benchmark, with ablations supporting our key design choices.

2 Related Work

Vision-language-action models (VLAs). Early large-scale VLAs [14, 15] introduce transformer-based policies trained on extensive robot trajectories paired with web-scale vision-language data. OpenVLA [2] open-sources a VLA pretrained on the Open X-Embodiment dataset [6]. More recently, π_0 [1] and GR00T N1 [3] combine a pretrained vision-language backbone with a diffusion-based action expert for continuous, high-frequency control. However, these VLAs still require substantial task- and embodiment-specific teleoperation data for post-training, motivating our focus on one-shot demonstration videos.

One-shot demo-conditioned policies. One line of work uses kinematic retargeting between humans and robots to learn from one-shot demonstrations [7, 8, 9]. These approaches enable explicit cross-embodiment transfer from human demonstrations to robot actions, but often rely on substantial task- or embodiment-specific engineering. We instead focus on end-to-end approaches that condition robot policies directly on demonstrations without explicit retargeting. XSkill [16] and UniSkill [11] learn cross-embodiment skill representations from human and robot videos, while Vid2Robot [10] trains an end-to-end demo-conditioned policy with video- and language-alignment objectives. ViVLA [12] learns a unified latent action space across human and robot demonstrations and trains a VLA to predict both latent action tokens and robot actions. While these methods have shown promise, we find that they can still struggle when task success depends on precisely localizing small interaction regions (§5.3). SeeTraceAct addresses this limitation through an auxiliary visual trace objective that encourages precise spatial grounding.

Visual traces in VLAs. Several works have incorporated visual traces into VLAs: LLARVA [17] and MolmoAct [18] predict future visual traces and feed them autoregressively to the action model, while RT-Trajectory [19] uses trajectory sketches obtained from human drawings, videos, or model predictions. HAMSTER [20] uses 2D traces as an intermediate representation in hierarchical policies, and TraceVLA [21] conditions the policy on past traces extracted from previous observations. Closer to our approach, ThinkAct [22] and FastThinkAct [23] condition action generation on visual latent plans rather than decoded visual traces. SeeTraceAct differs by using visual traces to ground demonstration videos into spatial plans while explicitly handling off-screen trace points.

3 Method

3.1 Problem setting

We consider a setting in which a policy is evaluated on unseen tasks, conditioned on a single demonstration video that may come from a different embodiment. Let $\mathcal{T}$ denote a distribution over robotic tasks, and let $\mathcal{T}_{\text{seen}}$ and $\mathcal{T}_{\text{unseen}}$ denote disjoint sets of seen and held-out unseen tasks sampled from this distribution. During training, for each seen task $\kappa \in \mathcal{T}_{\text{seen}}$, the policy π_θ has access to multiple training tuples $(\xi^\kappa, D^\kappa, l^\kappa)$, where $\xi^\kappa = \{(o_t, q_t, a_t)\}_{t=1}^{T}$ is an expert trajectory from the policy embodiment, D^κ is a demonstration video that may come from a different embodiment, and l^κ is the corresponding language instruction. Here, o_t, q_t, and a_t denote the camera view, robot state, and robot action at timestep t, respectively. We omit the task superscript for brevity.

At evaluation, the policy is given a single demonstration video D and language instruction l for an unseen task $\kappa^* \in \mathcal{T}_{\text{unseen}}$. At each timestep t, it takes the current observation (o_t, q_t) and predicts an action chunk $A_t = \{a_t, \ldots, a_{t+H-1}\}$ toward completing κ^*, where H denotes the action horizon: $A_t \sim \pi_\theta(\cdot \mid o_t, q_t, l, D)$.

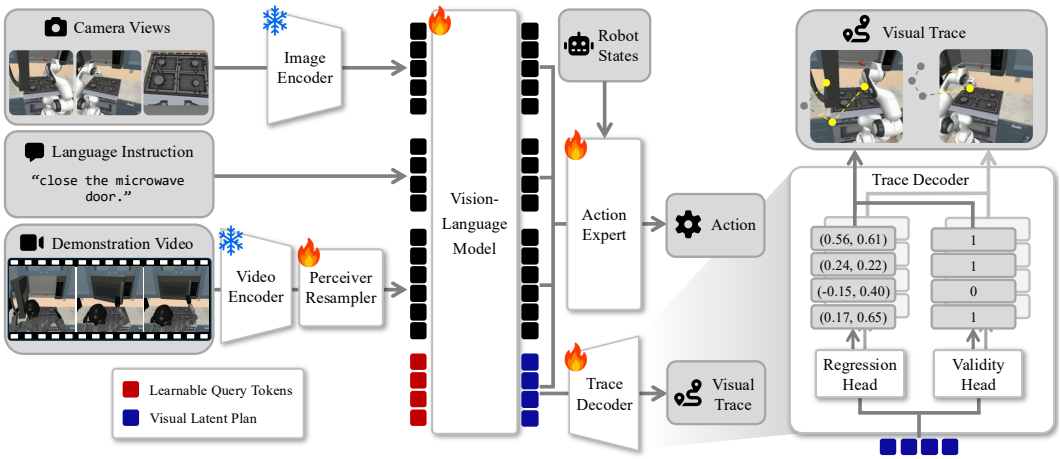


Figure 2: Architecture of SEETRACEACT. It receives camera views, a language instruction, a demonstration video, and robot states, and outputs an action chunk. We append learnable query tokens after the input tokens; their final hidden states form a *visual latent plan*, which is decoded into future end-effector traces during training. The trace decoder consists of a regression head that predicts the trace coordinates and a validity head that predicts whether each trace point is in-frame. The trace decoder is used only during training and discarded at inference time.

3.2 Architecture

We build SEETRACEACT on top of the GR00T N1.5 [3] architecture, which consists of a vision-language model (VLM) and a flow-matching-based action expert. As shown in Fig. 2, our model augments this architecture with video encoding modules, learnable query tokens that form a visual latent plan, and a trace decoder. At each control timestep t, the VLM processes image and language tokens from the current camera views and language instruction, producing a representation ϕ_t.

Video tokens. The left side of Fig. 2 shows how we incorporate the demonstration video. We first encode the demonstration D using a video encoder [24] pre-trained on large-scale video data, including action-centric datasets [25, 26]. We add a Perceiver Resampler [27, 28] on top of the video encoder to compress the extracted features into a compact set of video tokens (from 8,192 to 32 tokens; see App. B), and append them to the image and language tokens in the VLM input sequence. Under causal attention, this ordering lets the video tokens attend to the image and language tokens, making the demonstration representation context-dependent rather than fixed across the episode. We denote the final VLM hidden states corresponding to these video tokens by ψ_t.

Visual latent plan. The middle of Fig. 2 illustrates how we form the visual latent plan. Following recent visual latent planning work [22, 23], we use this term to refer to a latent representation that is trained to encode future task progression and conditions action generation. Rather than assigning this planning role directly to the existing input tokens, we append a separate set of learnable query tokens after them. Under causal attention, these query tokens attend to the image, language, and video tokens, and their final hidden states constitute the visual latent plan z_t.

Visibility-aware trace decoder. As shown on the right side of Fig. 2, the trace decoder predicts future end-effector traces in each static camera view from the visual latent plan z_t. However, the end effector may move off-screen in some camera views, making its normalized 2D image coordinates ill-posed regression targets. To address this, we use a *visibility-aware* trace decoder with two heads. The regression head predicts normalized end-effector coordinates, while the validity head predicts whether or not each trace point is in-frame, allowing off-screen points to still provide a learning signal (see ablation in §5.4). The trace decoder is discarded after training.

3.3 Training and inference

We train SEETRACEACT with two objectives: the *action prediction loss* and the auxiliary *visual trace loss*. The former optimizes the policy to predict robot action chunks, while the latter shapes the visual latent plan z_t to encode future task progression in the robot's image space.

Action prediction loss. Following prior works [1, 3], we train the action expert with an action prediction loss instantiated as a flow-matching objective [29]. At each control timestep t, we sample noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and draw a flow-matching timestep $\tau \in [0, 1]$ from a shifted beta distribution. Given the ground-truth action chunk A_t, we construct the noised action chunk as $A_t^\tau = \tau A_t + (1 - \tau)\epsilon$. The action prediction loss trains the action expert V_θ to predict the target velocity field from the VLM outputs ϕ_t, ψ_t, and z_t, together with q_t and the noised action chunk A_t^τ:

$$\mathcal{L}_{\text{act}}(\theta) = \mathbb{E}_{\tau,\epsilon}\left[\left\|V_\theta(\phi_t, \psi_t, z_t, A_t^\tau, q_t) - (A_t - \epsilon)\right\|^2\right]. \tag{1}$$

Visual trace loss. We supervise the policy with visibility-aware 2D image-space traces, directly aligning the target with the robot's visual observation space. For each camera view, we predict N future trace points at a temporal stride Δ using N learnable query tokens. We denote the final hidden states of these tokens by $z_t = \{z_{t,1}, \ldots, z_{t,N}\}$. The corresponding trace target is $Y_t = \{y_{t+\Delta}, y_{t+2\Delta}, \ldots, y_{t+N\Delta}\}$, where each $z_{t,n}$ is supervised to predict $y_{t+n\Delta}$, the relative coordinate of the end effector. We also define the binary validity target $M_t = \{m_{t+\Delta}, m_{t+2\Delta}, \ldots, m_{t+N\Delta}\}$, where $m_{t+n\Delta}$ indicates whether the end effector is in-frame. The trace decoder takes z_t as input and applies two heads. The regression head outputs the predicted trace $\hat{Y}_t = \{\hat{y}_{t+\Delta}, \ldots, \hat{y}_{t+N\Delta}\}$, while the validity head outputs the predicted validity scores $\hat{M}_t = \{\hat{m}_{t+\Delta}, \ldots, \hat{m}_{t+N\Delta}\}$. We define the masked regression loss as:

$$\mathcal{L}_{\text{reg}}(\theta) = \frac{1}{\max(1, \sum_{n=1}^{N} m_{t+n\Delta})} \sum_{n=1}^{N} m_{t+n\Delta} \left\|\hat{y}_{t+n\Delta} - y_{t+n\Delta}\right\|_1. \tag{2}$$

We define the validity loss with binary cross-entropy:

$$\mathcal{L}_{\text{valid}}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \text{BCE}\left(\hat{m}_{t+n\Delta}, m_{t+n\Delta}\right). \tag{3}$$

The auxiliary visual trace loss combines the regression and validity losses:

$$\mathcal{L}_{\text{trace}}(\theta) = \mathcal{L}_{\text{reg}}(\theta) + \lambda_{\text{valid}} \mathcal{L}_{\text{valid}}(\theta), \tag{4}$$

where λ_{valid} controls the relative weight of the validity loss. This loss is applied independently to each camera view and averaged across views.

Overall training objective. Our final training objective combines the action prediction loss and the visual trace loss:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{act}}(\theta) + \lambda_{\text{trace}} \mathcal{L}_{\text{trace}}(\theta), \tag{5}$$

where λ_{trace} controls the strength of the visual trace supervision.

Inference. During inference, the VLM backbone processes the camera views, language instruction, and demonstration video to obtain the representations ϕ_t, ψ_t, and z_t at control timestep t. The action expert predicts actions from ϕ_t, ψ_t, z_t, q_t, and the current noisy action estimate. We initialize $A_t^0 = \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and update it with Euler integration: $A_t^{(k+1)/K} = A_t^{k/K} + \frac{1}{K} V_\theta(\phi_t, \psi_t, z_t, A_t^{k/K}, q_t)$ for $k = 0, \ldots, K-1$. After K denoising steps, we obtain A_t^1 as the final predicted action chunk.

4 RoboCasa-DC

Base environment. RoboCasa [13] provides a suite of 24 kitchen manipulation tasks with a 7-DoF Panda arm. Each observation consists of proprioceptive states and three camera views, including two static cameras and one wrist-mounted camera. The original benchmark provides approximately

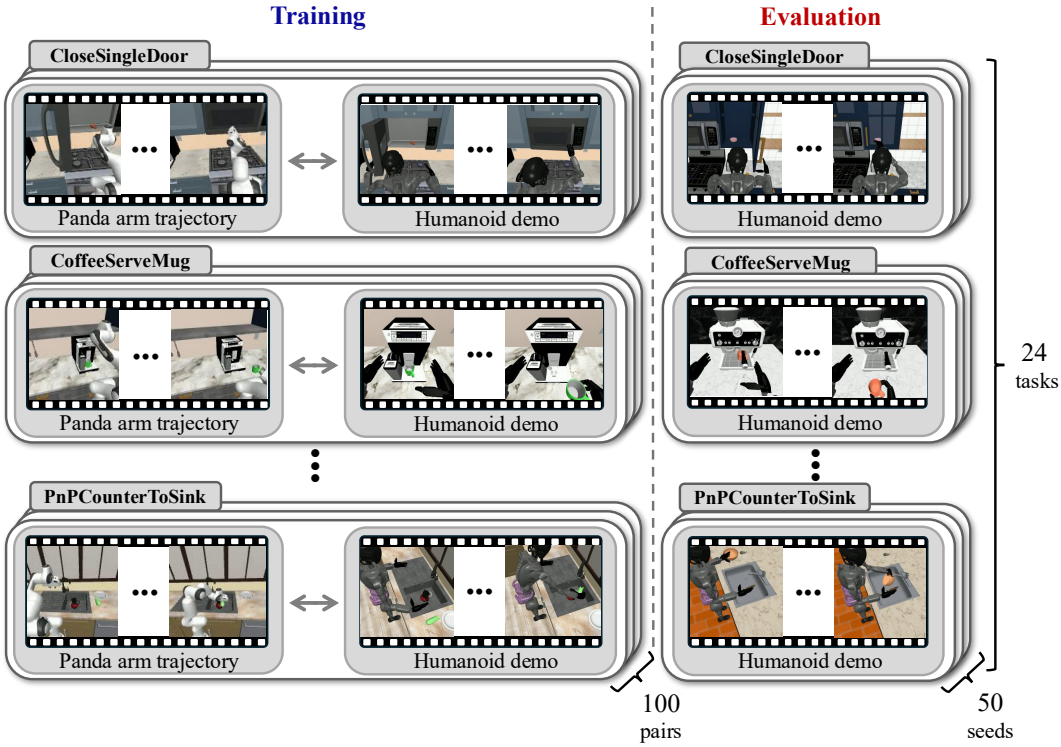


Figure 3: Cross-embodiment benchmark dataset in RoboCasa-DC. For each of the 24 tasks, we pair 100 original Panda-arm trajectories with collected GR-1 humanoid demonstrations for training. For evaluation, we collect humanoid demonstrations for 50 pre-defined seeds per task.

3,000 expert Panda-arm trajectories per task for training, and evaluates trained policies on unseen seeds that vary scene layouts, object categories, and object placements.

Benchmark construction. RoboCasa-DC extends RoboCasa with demonstration videos that share the task and initial scene state with each Panda-arm trajectory, but may differ in embodiment and trajectory, allowing policies to condition on either cross- or same-embodiment demonstrations. Fig. 3 illustrates the cross-embodiment case, where Panda-arm trajectories are paired with corresponding GR-1 humanoid demonstrations. For this setting, we pair 100 Panda-arm trajectories per task with corresponding humanoid expert demonstrations across all 24 tasks. We collect each humanoid demonstration by restoring the corresponding Panda-arm trajectory's initial state and teleoperating the humanoid in the same scene with a Leap Motion controller, providing a simulated proxy for real-world human demonstrations. For evaluation, we pre-define 50 evaluation seeds per task and collect corresponding humanoid demonstrations for each seed. For each task, we manually select the demonstration camera view that best captures task execution. We also include a same-embodiment case, following prior work [11, 12], where videos from Panda-arm trajectories are used as demonstrations. We publicly release RoboCasa-DC to support reproducible evaluation and future work on demo-conditioned policies.

Evaluation protocol. A RoboCasa-DC evaluation split partitions the 24 tasks into seen training tasks and unseen evaluation tasks. Demo-conditioned policies are trained on the paired dataset from the seen tasks. At test time, each policy is evaluated on evaluation seeds from unseen tasks while conditioning on the single demonstration video collected for each seed, which shares the same initial scene state. This protocol tests whether a policy can extract task-relevant guidance from one-shot demonstrations, including cross-embodiment ones, rather than merely imitating expert trajectories from its own embodiment.

5 Experiments

5.1 Experimental setup

RoboCasa-DC. We evaluate methods on two task splits of RoboCasa-DC: a *category-balanced* split (similar to [30]) and a *precision-sensitive* split (motivated by [31, 32]). Each split holds out five tasks as unseen evaluation tasks, while the remaining 19 tasks are used for training. The category-balanced split samples unseen tasks from different action categories: *CloseDrawer*, *TurnOffSinkFaucet*, *OpenDoubleDoor*, *CoffeeServeMug*, and *PnPCounterToMicrowave*. The precision-sensitive split instead holds out five tasks with small target interaction ratio (TIR) (see Fig. 7), which stress precise spatial grounding: *TurnOffStove*, *CoffeePressButton*, *TurnOffSinkFaucet*, *PnPCounterToSink*, and *PnPStoveToCounter*. We use the dataset and evaluation protocol described in §4. For SeeTraceAct, the trace labels are generated automatically by projecting end-effector positions into each static camera view.

Real-world benchmark. We evaluate demo-conditioned policies on real-world tabletop manipulation tasks using a Franka Panda arm (see Fig. 6 in App. A.2). For each of four seen tasks—*Pick Coke*, *Stack Blocks*, *Stack Cups*, and *Close Laptop*—we collect 150 episode pairs, each consisting of a teleoperated Franka trajectory and a human demonstration performed from the same initial scene; policies are trained on these pairs. For SeeTraceAct, we obtain trace labels from the robot trajectories during collection by projecting end-effector positions into the static camera view, following [19, 20]. For evaluation, we define six unseen tasks: *Pick Block*, *Pick Cup*, *Press Button*, *Stack Blocks in Swapped Order*, *Place Block in Bin*, and *Close Kettle Lid*. These tasks are designed by recombining seen skills and objects, transferring seen skills to novel objects, and introducing a new contact interaction. We test each unseen task across 10 different object positions and poses. Following prior evaluation setups [11, 12, 16], each policy is conditioned on a one-shot human demonstration video captured in a closely matched scene. We report per-task and average success rates over the unseen tasks, further averaged over three independent runs. Hardware setup and task language instructions are provided in Fig. 5 and Table 3, respectively.

Baselines. We compare SeeTraceAct against three demo-conditioned baselines: Vid2Robot [10], UniSkill [11], and ViVLA [12]. For fair comparison, all baselines are re-implemented on top of the same GR00T N1.5 backbone as SeeTraceAct and trained with their corresponding objectives. Further implementation details are provided in App. B.

5.2 Results

Table 1 summarizes the experimental results on RoboCasa-DC. SeeTraceAct achieves the best performance across all four evaluation settings, covering both category-balanced and precision-sensitive splits under same-embodiment and cross-embodiment demonstrations. This consistency is important because the two axes stress different aspects of demo-conditioned control: the precision-sensitive split requires fine-grained spatial grounding, while the cross-embodiment setting requires extracting task-relevant motion from demonstrations whose embodiment differs from the robot. Although absolute performance generally decreases under these harder settings, SeeTraceAct maintains its advantage over the baselines, with the largest margin appearing in the precision-sensitive cross-embodiment setting.

Fig. 4 shows the experimental results on the real-world benchmark. SeeTraceAct again matches or outperforms every baseline on all six unseen tasks, improving average success from 33.3% to 42.8% over the strongest baseline. These results extend the gains observed in simulation to real-world cross-embodiment demo conditioning.

5.3 Larger gains on precision-sensitive tasks

Table 1 shows that SeeTraceAct's margin over the strongest baseline is larger on the precision-sensitive split than on the category-balanced split, increasing from 0.7 to 2.6 percentage points on

Method	Category-balanced split		Precision-sensitive split	
	Same-emb demo	Cross-emb demo	Same-emb demo	Cross-emb demo
Vid2Robot [10]	17.9%	9.5%	11.4%	5.6%
UniSkill [11]	13.0%	11.3%	11.7%	5.6%
ViVLA [12]	14.1%	9.1%	8.8%	7.2%
SEETRACEACT (Ours)	**18.9%**	**11.6%**	**13.2%**	**10.8%**

Table 1: Experimental results on ROBOCASA-DC. We report success rates averaged over the five unseen tasks in each split, with 50 evaluation episodes per task, and further averaged over three independent runs. In *Same-emb demo*, policies are conditioned on demonstrations from the same Panda-arm embodiment; in *Cross-emb demo*, policies are conditioned on simulated GR-1 humanoid demonstrations as a proxy for human videos.

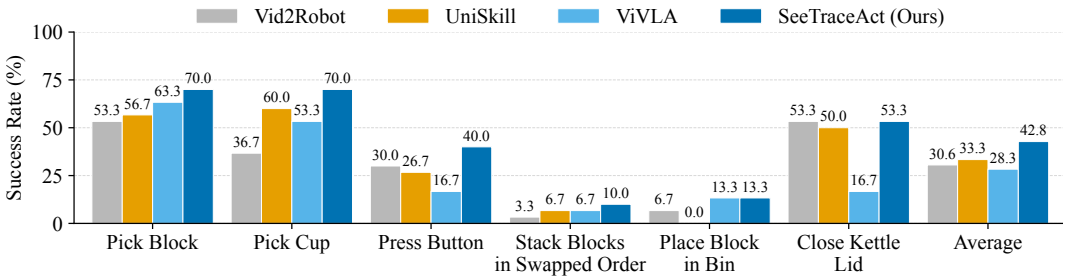


Figure 4: Experimental results on the real-world benchmark with a Franka Panda arm. We report success rates over 10 trials for each of the six unseen tasks, averaged over three independent runs, along with the average across tasks.

average across the same- and cross-embodiment settings. This suggests that visibility-aware trace supervision is especially useful when successful execution depends on accurately localizing a small task-critical interaction region.

To examine this trend more directly, we train all models on all 24 ROBOCASA-DC tasks and evaluate them on 50 held-out seeds per task (Table 5), similar to [31]. We use the same-embodiment demonstrations to factor out the cross-embodiment gap. In this setting, SEETRACEACT again achieves the highest average success rate. To characterize where these gains arise, we measure precision sensitivity using the target interaction ratio (TIR), defined as the fraction of the image covered by the target interaction region (Fig. 7); a smaller TIR indicates a more precision-sensitive task. We compare TIR with the per-task improvement of SEETRACEACT over each baseline using a one-sided Spearman rank correlation test. Across the 24 tasks, TIR is negatively correlated with the gain over every baseline: Vid2Robot ($\rho = -0.80$, $p < 10^{-5}$), UniSkill ($\rho = -0.45$, $p < 0.05$), and ViVLA ($\rho = -0.52$, $p < 0.01$). The correlation remains significant even when we compare SEETRACEACT against the strongest baseline for each task ($\rho = -0.63$, $p < 10^{-3}$), consistent with the larger gains on precision-sensitive tasks in Table 1.

5.4 Ablation study

We ablate the design choices of SEETRACEACT in the cross-embodiment setting of ROBOCASA-DC, averaging results across the category-balanced and precision-sensitive splits. We also report the GR00T N1.5 backbone without demonstrations, with and without our trace supervision, as reference points for standard VLA policies. Table 2 shows that neither demonstration inputs nor trace supervision is sufficient on its own. Removing trace supervision from SEETRACEACT reduces performance to 9.3%, while adding trace supervision to the no-demonstration backbone yields only 9.0%. Removing the validity head lowers performance to 8.6%, below even the variant without any trace supervision. This supports the importance of visibility-aware supervision; forcing the model to regress ill-defined off-screen coordinates can hurt representation learning. Replacing the V-JEPA

Method	Success rate
GR00T N1.5 backbone (no demonstrations)	8.5%
GR00T N1.5 backbone w/ trace supervision (no demonstrations)	9.0%
SEETRACEACT w/o trace supervision	9.3%
SEETRACEACT w/o validity head	8.6%
SEETRACEACT w/o action-aware video encoder	8.1%
SEETRACEACT w/ 3D trace supervision	9.7%
SEETRACEACT w/ demonstration trace supervision	8.7%
SEETRACEACT (Ours)	**11.2%**

Table 2: Ablation study in the cross-embodiment setting of ROBOCASA-DC. Success rates are averaged over the category-balanced and precision-sensitive splits, and further averaged over three independent runs.

2 [24] video encoder with the backbone VLA's SigLIP [33] encoder also lowers performance to 8.1%, suggesting that an action-aware video representation is useful for extracting task progression from demonstrations. We further consider two alternative trace targets. For 3D trace supervision, we append to each 2D point the metric depth along the viewing direction, computed from the logged end-effector positions and camera calibration. This variant remains the strongest ablation but still lags behind image-space supervision, likely because 2D traces across multiple views already constrain the 3D position, so adding depth offers little new information while introducing a target that the RGB-pretrained backbone must infer implicitly. Finally, supervising traces of the demonstrator's end effector in the demonstration video, instead of the robot's, lowers performance to 8.7%.

6 Limitations

Our study has several limitations. First, our real-world evaluation is limited to a tabletop single-arm benchmark, leaving broader evaluation across embodiments, environments, and tasks for future work. Second, our auxiliary objective assumes future trace labels during training; while they can be computed from end-effector positions and camera calibration already provided by many robot datasets [5], when these are unavailable, such labels could be obtained using motion trackers or foundation models, following prior work [17, 21, 22, 23, 18, 19]. Finally, despite improving over baselines, the modest absolute success rates on ROBOCASA-DC show that demo-conditioned robot learning leaves substantial room for progress.

7 Conclusion

We presented SEETRACEACT, a demo-conditioned VLA framework that grounds one-shot demonstrations through visibility-aware latent planning. We also introduced ROBOCASA-DC, a benchmark for evaluating demo-conditioned policies with same- and cross-embodiment demonstrations. Experiments show that SEETRACEACT achieves the strongest performance among baselines, with especially large gains on precision-sensitive tasks. These results highlight precise spatial grounding as a promising direction for demo-conditioned robot learning.

Acknowledgments

We thank Jaeah Lee, Hanjung Kim, and the anonymous reviewers for their insightful feedback, and Junghwan Yim for data collection in the course project that preceded this work. This material is based upon work partially supported by the National Science Foundation under Grant No. 2239292 and NVIDIA Academic Grant Program.

References

[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. $\pi 0$: A vision-language-action flow model for general robot control. *ArXiv*, abs/2410.24164, 2024.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, G. Lam, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. *ArXiv*, abs/2406.09246, 2024.

[3] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z.-T. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *ArXiv*, abs/2503.14734, 2025.

[4] H. R. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. H. Vuong, A. W. He, V. Myers, K. Fang, C. Finn, and S. Levine. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023.

[5] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. Ma, P. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J.-P. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. B. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Zhao, C. Agia, R. Baijal, M. G. Castro, D. L. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, M. Z. Irshad, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. M. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Mart'in-Mart'in, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[6] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, A. Tung, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Gupta, A. Wang, A. Singh, A. Garg, A. Kembhavi, A. Xie, A. Brohan, A. Raffin, A. Sharma, A. Yavary, A. Jain, A. Balakrishna, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Wulfe, B. Ichter, C. Lu, C. Xu, C. Le, C. Finn, C. Wang, C. Xu, C. Chi, C. Huang, C. Chan, C. Agia, C. Pan, C. Fu, C. Devin, D. Xu, D. Morton, D. Driess, D. Chen, D. Pathak, D. Shah, D. Büchler, D. Jayaraman, D. Kalashnikov, D. Sadigh, E. Johns, E. P. Foster, F. Liu, F. Ceola, F. Xia, F. Zhao, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Feng, G. Schiavi, G. Berseth, G. Kahn, G. Wang, H. Su, H. Fang, H. Shi, H. Bao, H. B. Amor, H. I. Christensen, H. Furuta, H. R. Walke, H. Fang, H. Ha, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Abou-Chakra, J. Kim, J. Drake, J. Peters, J. Schneider, J. Hsu, J. Bohg, J. Bingham, J. Wu, J. Gao, J. Hu, J. Wu, J. Wu, J. Sun, J. Luo, J. Gu, J. Tan, J. Oh, J. Wu, J. Lu,

J. Yang, J. Malik, J. Silvério, J. Hejna, J. Booher, J. Tompson, J. Yang, J. Salvador, J. J. Lim, J. Han, K. Wang, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Black, K. Lin, K. Zhang, K. Ehsani, K. Lekkala, K. Ellis, K. Rana, K. P. Srinivasan, K. Fang, K. P. Singh, K.-H. Zeng, K. B. Hatch, K. Hsu, L. Itti, L. Y. Chen, L. Pinto, F.-F. Li, L. Tan, L. J. Fan, L. Ott, L. Lee, L. Weihs, M. Chen, M. Lepert, M. Memmel, M. Tomizuka, M. Itkina, M. G. Castro, M. Spero, M. Du, M. Ahn, M. C. Yip, M. Zhang, M. Ding, M. Heo, M. K. Srirama, M. Sharma, M. J. Kim, M. Z. Irshad, N. Kanazawa, N. Hansen, N. M. O. Heess, N. J. Joshi, N. Suenderhauf, N. Liu, N. D. Palo, N. M. Shafiullah, O. Mees, O. Kroemer, O. Bastani, P. R. Sanketi, P. Miller, P. Yin, P. Wohlhart, P. Xu, P. D. Fagan, P. Mitrano, P. Sermanet, P. Abbeel, P. Sundaresan, Q. Chen, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Baijal, R. Scalise, R. Hendrix, R. Lin, R. Qian, R. Zhang, R. Mendonca, R. Shah, R. Hoque, R. C. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Lin, S. Moore, S. Bahl, S. Dass, S. D. Sonawani, S. Song, S. Xu, S. Haldar, S. Karamcheti, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Ramamoorthy, S. Dasari, S. Belkhale, S. Park, S. Nair, S. Mirchandani, T. Osa, T. Gupta, T. Harada, T. Matsushima, T. Xiao, T. Kollar, T. Yu, T. Ding, T. Davchev, T. Zhao, T. Armstrong, T. Darrell, T. Chung, V. Jain, V. Vanhoucke, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Wang, X. Zhu, X. Geng, X. Liu, L. Xu, X. Li, Y. Lu, Y. J. Ma, Y. Kim, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Wu, Y. Xu, Y. Wang, Y. Bisk, Y. Cho, Y. Lee, Y. Cui, Y. Cao, Y.-H. Wu, Y. Tang, Y. Zhu, Y. Zhang, Y. Jiang, Y. Li, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Ma, Z. Xu, Z. J. Cui, Z. Zhang, and Z. Lin. Open x-embodiment: Robotic learning datasets and rt-x models : Open x-embodiment collaboration. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[7] S. Park, H. Bharadhwaj, and S. Tulsiani. Demodiffusion: One-shot human imitation using pre-trained diffusion policy. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[8] J. Li, Y. Zhu, Y. Xie, Z. Jiang, M. Seo, G. Pavlakos, and Y. Zhu. Okami: Teaching humanoid robots manipulation skills through single video imitation. In *Conference on Robot Learning (CoRL)*, 2024.

[9] N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada. Ditto: Demonstration imitation by trajectory transformation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[10] V. Jain, M. Attarian, N. J. Joshi, A. Wahid, D. Driess, Q. Vuong, P. R. Sanketi, P. Sermanet, S. Welker, C. Chan, I. Gilitschenski, Y. Bisk, and D. Dwibedi. Vid2robot: End-to-end video-conditioned policy learning with cross-attention transformers. In *Robotics: Science and Systems (RSS)*, 2024.

[11] H. Kim, J. Kang, H. Kang, M. Cho, S. J. Kim, and Y. Lee. Uniskill: Imitating human videos via cross-embodiment skill representations. In *Conference on Robot Learning (CoRL)*, 2025.

[12] G. Chen, M. Wang, Q. Shao, Z. Zhou, W. Mao, T. Cui, M. Zhu, Y. Deng, L. Yang, Z. Zhang, Y. Yang, H. Chen, and Y. Yue. See once, then act: Vision-language-action model with task learning from one-shot video demonstrations. *ArXiv*, abs/2512.07582, 2025.

[13] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[14] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. C. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. S. Ryoo, G. Salazar, P. R. Sanketi, K. Sayed, J. Singh, S. A. Sontakke, A. Stone,

C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. H. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023.

[15] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, K. Choromanski, T. Ding, D. Driess, K. A. Dubey, C. Finn, P. R. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. J. Joshi, R. C. Julian, D. Kalashnikov, Y. Kuang, I. Leal, S. Levine, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. S. Ryoo, G. Salazar, P. R. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. H. Vuong, A. Wahid, S. Welker, P. Wohlhart, T. Xiao, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

[16] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song. Xskill: Cross embodiment skill discovery. In *Conference on Robot Learning (CoRL)*, 2023.

[17] D. Niu, Y. Sharma, G. Biamby, J. Quenum, Y. Bai, B. Shi, T. Darrell, and R. Herzig. Llarva: Vision-action instruction tuning enhances robot learning. In *Conference on Robot Learning (CoRL)*, 2024.

[18] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. Molmoact: Action reasoning models that can reason in space. *ArXiv*, abs/2508.07917, 2025.

[19] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. Gonzalez Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu, P. Sundaresan, P. Xu, H. Su, K. Hausman, Q. Vuong, and T. Xiao. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. In *International Conference on Learning Representations (ICLR)*, 2024.

[20] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, R. Yu, C. R. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta, and A. Goyal. Hamster: Hierarchical action models for open-world robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[21] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daum'e, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations (ICLR)*, 2025.

[22] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. In *Neural Information Processing Systems (NeurIPS)*, 2025.

[23] C.-P. Huang, Y. Man, Z. Yu, M.-H. Chen, J. Kautz, Y.-C. F. Wang, and F.-E. Yang. Fast-thinkact: Efficient vision-language-action reasoning via verbalizable latent planning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[24] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. Robert Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv*, abs/2506.09985, 2025.

[25] R. Goyal, S. E. Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fründ, P. N. Yianilos, M. Mueller-Freitag, F. Hoppe, C. Thurau, I. Bax, and R. Memisevic. The "something something" video database for learning and evaluating visual common sense. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2017.

[26] W. Kay, J. Carreira, K. Simonyan, B. H. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, A. Natsev, M. Suleyman, and A. Zisserman. The kinetics human action video dataset. *ArXiv*, abs/1705.06950, 2017.

[27] J. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan. Flamingo: a visual language model for few-shot learning. In *Neural Information Processing Systems (NeurIPS)*, 2022.

[28] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021.

[29] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[30] T. Yu, D. Quillen, Z. He, R. C. Julian, K. Hausman, C. Finn, and S. Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2019.

[31] A. Goyal, V. Blukis, J. Xu, Y. Guo, Y.-W. Chao, and D. Fox. Rvt2: Learning precise manipulation from few demonstrations. In *Robotics: Science and Systems (RSS)*, 2024.

[32] Y. Yin, Z. Han, S. Aarya, S. Xu, J. Wang, J. Peng, A. Wang, A. Yuille, and T. Shu. Partinstruct: Part-level instruction following for fine-grained robot manipulation. In *Robotics: Science and Systems (RSS)*, 2025.

[33] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[34] S. Ye, J. Jang, B. Jeon, S. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin, et al. Latent action pretraining from videos. In *International Conference on Learning Representations (ICLR)*, 2025.

A Benchmark Details

A.1 RoboCasa-DC

To collect GR-1 humanoid demonstrations, we restore the initial simulation state of each pre-defined Panda-arm trajectory and teleoperate the humanoid in the same scene. Because the released RoboCasa trajectories do not include the generative fixture textures needed to exactly restore the original rendered observations, we randomly assign fixture textures, replay the Panda-arm trajectories, and re-render the corresponding videos. For the cross-embodiment setting, we use the paired Panda–GR-1 episodes described in §4 for training and the pre-defined evaluation seeds for testing. For the same-embodiment case, demonstrations are rendered from the Panda-arm trajectories themselves, allowing us to use the full RoboCasa training set, up to approximately 3,000 trajectories per task.

A.2 Real-world benchmark

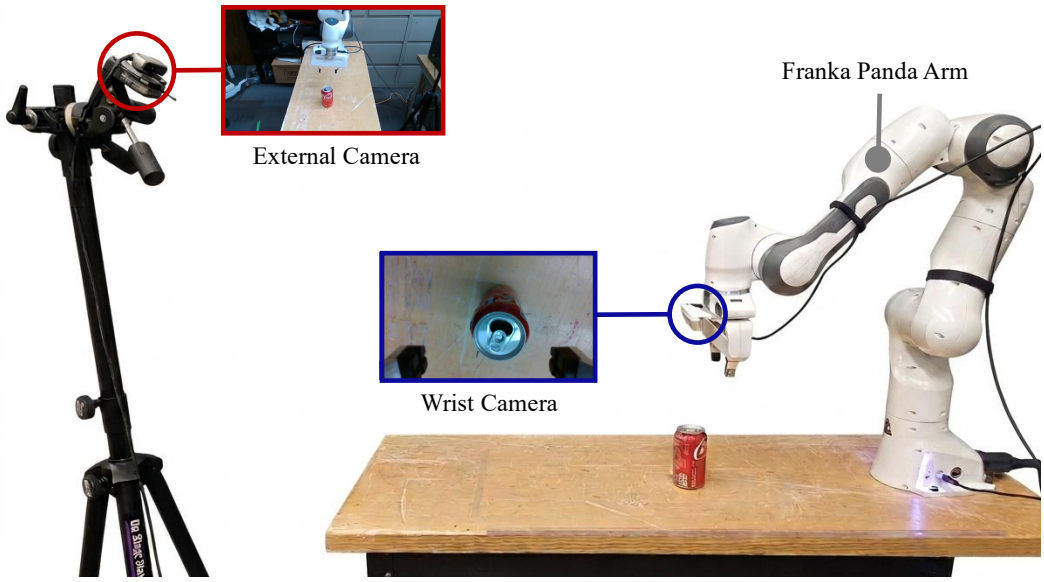


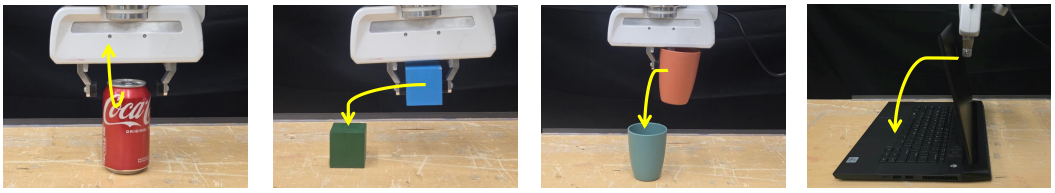
Figure 5: Hardware setup for the real-world experiments. The setup consists of a Franka Panda arm, an external third-person-view camera, and a wrist-mounted camera.

Hardware setup. Our real-world experiments use a 7-DoF Franka Panda arm mounted on a tabletop and equipped with a parallel-jaw gripper (Fig. 5). The visual observations are captured from two RGB cameras: a single external camera providing the third-person view and a wrist-mounted camera attached near the end effector. Both camera streams are captured at a resolution of 640×480.

Tasks. The language instructions for the real-world tasks shown in Fig. 6 are listed in Table 3.

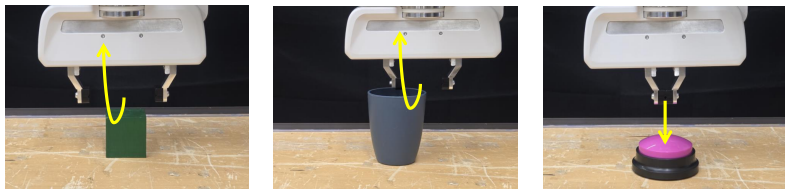
B Model Details

Common setup. For fair comparison, all methods are implemented on top of the same GR00T N1.5 architecture. Each method receives the same camera views, language instruction, demonstration video, and robot states as input, and predicts action chunks with the same horizon. Each baseline method is trained with the corresponding objectives proposed in the original work. For action prediction, we use an action horizon of $H = 16$ and perform $K = 4$ denoising steps at inference time. For the action prediction loss, we sample $u \sim \mathrm{Beta}(1.5, 1.0)$ and set $\tau = (s-u)/s$ with $s = 0.999$, following prior work [1, 3]. The hyperparameters used in our experiments are summarized in Table 4. Further details are provided in our code.

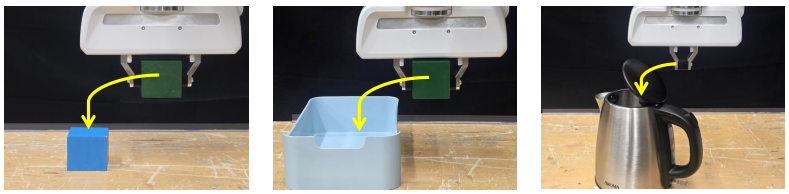


Pick Coke | Stack Blocks | Stack Cups | Close Laptop

(a) Seen tasks



Pick Block | Pick Cup | Press Button



Stack Blocks in Swapped Order | Place Block in Bin | Close Kettle Lid

(b) Unseen tasks

Figure 6: (a) Four seen tasks and (b) six unseen tasks in the real-world benchmark. The yellow arrow indicates the desired path of the end effector.

Task	Language instruction
Seen tasks	
Pick Coke	"Grab the coke can and lift it up"
Stack Blocks	"Stack the blue block on top of the green block"
Stack Cups	"Stack the orange cup onto the green cup"
Close Laptop	"Close the laptop lid fully"
Unseen tasks	
Pick Block	"Grab the green block and lift it up"
Pick Cup	"Grab the blue cup and lift it up"
Press Button	"Press the pink button"
Stack Blocks in Swapped Order	"Stack the green block on top of the blue block"
Place Block in Bin	"Place the block into the bin"
Close Kettle Lid	"Close the kettle lid fully"

Table 3: Real-world benchmark tasks and corresponding language instructions.

SeeTraceAct. For demonstration video encoding, we follow the open-sourced V-JEPA 2 preprocessing and uniformly sample 64 frames from each demonstration video. The sampled frames are resized to the video encoder input resolution of 256×256. Directly feeding all video features into the VLM would substantially increase the input sequence length. With a 3D patch size of $2 \times 16 \times 16$, a 64-frame video produces $32 \times 16 \times 16 = 8192$ video tokens, which is much larger than the approximately 200 image and language tokens used by the backbone. We therefore employ a Perceiver Resampler to compress the video features into 32 video tokens before appending them to the VLM input sequence. As in the original GR00T N1.5, robot observation images are encoded with its SigLIP [33] image encoder.

Hyperparameter	Vid2Robot	UniSkill	ViVLA	SeeTraceAct
Training				
Training steps	50,000			
Batch size	128			
Optimizer	AdamW (β_1=0.95, β_2=0.999, ϵ=10^{-8})			
Learning rate	10^{-4}			
Weight decay	10^{-5}			
Warmup steps	2500			
Learning rate scheduler	cosine			
Gradient clip norm	1.0			
Demonstration video				
Encoder architecture	SigLIP + Perceiver	ISD	V-JEPA 2 + Perceiver	
Frame sampling method	uniform	timestep	uniform	
Encoder input resolution	224×224	224×224	256×256	
# frames sampled	64	—	64	
# video tokens	32	1	32	
Action prediction				
Action horizon (H)	16			
Denoising steps (K)	4			
Trace supervision				
Validity loss weight (λ_{valid})	—	—	—	1.0
Visual trace loss weight (λ_{trace})	—	—	—	0.2
# future trace points (N)	—	—	—	5
Temporal stride (Δ)	—	—	—	5

Table 4: Hyperparameters used for the compared methods on RoboCasa-DC. *Encoder input resolution* follows each video encoder's default. Following [11], UniSkill extracts demonstration frames at timesteps t and $t+k$ at each control timestep, where k is the skill interval, while the other methods use uniformly sampled frames from the episode.

Vid2Robot [10]. Since Vid2Robot does not provide an official implementation, we re-implement the method described in the paper within our common GR00T N1.5 backbone. The original method uses a shared visual encoder for robot observations and prompt videos, so we use the SigLIP encoder from GR00T N1.5 to encode demonstration videos as well. We then pass the resulting video features through a Perceiver Resampler to compress them into 32 video tokens. We train the model with the action prediction loss, the prompt–robot video contrastive loss (VVCL), and the video–text contrastive loss (VTCL). In the same-embodiment setting, we omit VVCL because it already attains its theoretical minimum.

UniSkill [11]. We use the inverse skill dynamics (ISD) module from the official UniSkill implementation. At each control timestep t, UniSkill passes the demonstration frames at t and $t+k$ to the ISD module, where k is the skill interval, to obtain a skill representation for subsequent action prediction. Following the original implementation, we set $k = 20$ frames. We project the resulting skill representation into the VLM embedding space and append it as a single token to the input image, language, and video tokens.

ViVLA [12]. Because ViVLA does not provide an official implementation, we re-implement the method described in the paper within our common GR00T N1.5 backbone. To isolate the effect of the latent action prediction objective rather than the video encoder, we use the same V-JEPA 2 video encoder and Perceiver Resampler as those used in SeeTraceAct to generate input video tokens. For the latent action encoder, we use the open-sourced LAPA [34]. We remove the prediction of the number of LACT tokens and instead fix the number of tokens to 9, allowing ViVLA to generate each action chunk with a single forward pass. Specifically, we divide the demonstration video into 8 equal segments and train 8 LACT tokens to predict the LAPA representations for the corresponding segments, while the remaining LACT token predicts the representation of the robot's subsequent latent action. These 9 auxiliary tokens provide at least as much capacity as the 5 learnable query tokens used by SeeTraceAct.

Task	Vid2Robot	UniSkill	ViVLA	SeeTraceAct	TIR
CloseDoubleDoor	83.3	51.9	85.2	72.2	6.403
CloseDrawer	98.1	100.0	94.4	94.4	4.736
CloseSingleDoor	94.4	96.3	94.4	92.6	8.801
CoffeePressButton	64.8	37.0	70.4	83.3	0.092
CoffeeServeMug	53.7	57.4	74.1	53.7	0.507
CoffeeSetupMug	25.9	18.5	27.8	24.1	0.708
OpenDoubleDoor	20.8	13.2	41.5	17.0	1.349
OpenDrawer	33.3	48.1	53.7	48.1	0.281
OpenSingleDoor	57.4	29.6	55.6	61.1	0.439
PnPCabToCounter	42.6	51.9	44.4	50.0	0.427
PnPCounterToCab	33.3	11.1	22.2	33.3	0.671
PnPCounterToMicrowave	20.4	14.8	22.2	18.5	0.452
PnPCounterToSink	37.0	14.8	31.5	38.9	0.153
PnPCounterToStove	30.2	7.5	32.1	26.4	0.366
PnPMicrowaveToCounter	27.8	9.3	51.9	33.3	0.275
PnPSinkToCounter	7.4	7.4	18.5	22.2	0.165
PnPStoveToCounter	40.7	18.5	51.9	48.1	0.165
TurnOffMicrowave	79.6	48.1	74.1	83.3	0.262
TurnOffSinkFaucet	75.9	72.2	75.9	85.2	0.116
TurnOffStove	37.0	42.6	46.3	50.0	0.049
TurnOnMicrowave	64.8	63.0	68.5	66.7	0.470
TurnOnSinkFaucet	20.4	37.0	31.5	46.3	0.189
TurnOnStove	60.4	66.0	54.7	79.2	0.067
TurnSinkSpout	87.0	94.4	88.9	87.0	0.336
Average	49.9	42.1	54.7	54.8	1.139

Table 5: Per-task success rates when training all methods on all 24 RoboCasa-DC tasks. Results are reported in the same-embodiment setting, with evaluation on 50 held-out seeds per task. We also report the target interaction ratio (TIR), defined as the area ratio of the target interaction region to the full camera view, as visualized in Fig. 7. Lower TIR indicates higher precision sensitivity. Success rates and TIR are reported as percentages.

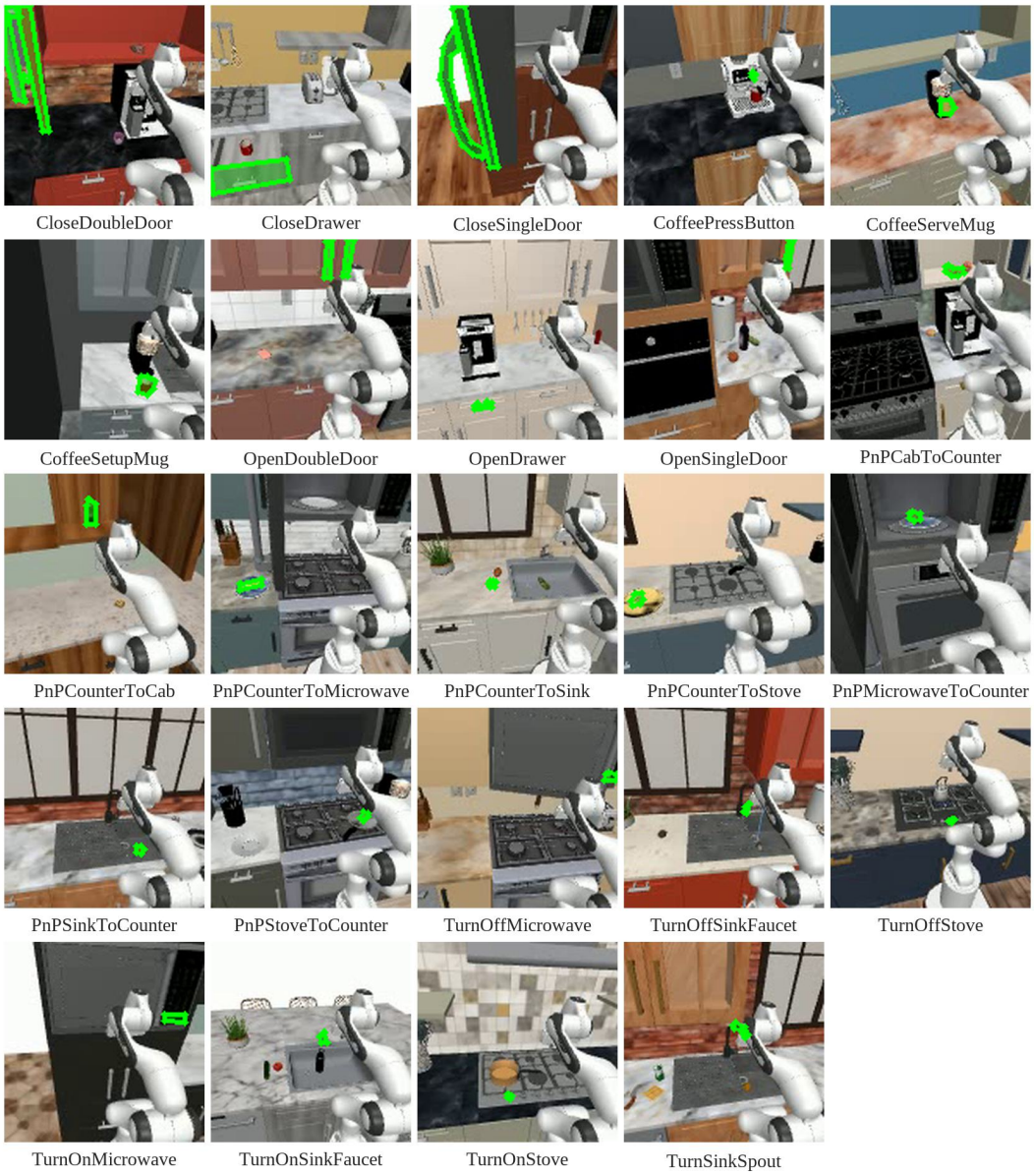


Figure 7: Target interaction regions in RoboCasa-DC tasks. The highlighted region indicates the area in a static camera view where the robot must interact to complete the task. We use the area ratio of this region to the full camera view as the *target interaction ratio* (TIR) in §5 and Table 5; lower TIR indicates a more precision-sensitive task.